

TCC · ORGANIZAÇÃO DO SISTEMA · 30 PÁGINAS

Controla.AI — Organização Completa do Sistema

Pastas · Código · Produção

Documento didático: o que cada pasta faz, como WhatsApp, agente de IA, indicadores e banco se conectam, e como o código sobe para **Railway** (API/VPS) e **Vercel** (painel web).

ESTRUTURA

Págs. 2–3 — Sumário e visão geral
Págs. 4–8 — Arquitetura e pastas
Págs. 9–16 — Backend, IA, WhatsApp
Págs. 17–21 — Frontend, indicadores, banco
Págs. 22–28 — Fluxos e produção Railway/Vercel
Págs. 29–30 — Mapa de arquivos e conclusão

EQUIPE

Davi Almeida
Leonardo Sena
Gustavo Biscoto

Setembro / 2026

Curitiba — PR



Sumário

Navegação das 30 páginas

Pág.	Conteúdo
1	Capa
2	Sumário
3	Visão geral do produto
4	Arquitetura em camadas
5	Diagrama de conexões (canais → API → banco)
6	Árvore de pastas (raiz)
7	Pasta backend/ — visão
8	Pasta frontend/ e documentacao-tcc/
9	Servidor Fastify (src/index.ts)
10	Auth, mailer, rotas CRUD
11	Agente financeiro unificado
12	Parser OpenAI, prompts, mídia
13	Insights e indicadores
14	WhatsApp Baileys — conexão
15	WhatsApp — pipeline de mensagem
16	Onboarding, renda e metas
17	Frontend React — boot e rotas
18	Dashboard, chat, admin
19	Funções Vercel (auth / proxy)
20	Banco PostgreSQL e Drizzle
21	Tabelas principais e FKs
22	Fluxo: gasto pelo WhatsApp
23	Fluxo: login e painel web
24	Produção: Railway (API + Postgres + Redis)
25	Produção: Vercel (frontend + proxy)
26	Como o código se liga ao servidor
27	Variáveis de ambiente
28	Deploy e health checks
29	Mapa de arquivos comentados
30	Conclusão e referências

Fonte oficial

Este HTML complementa **TCC_DOCUMENTACAO.md** (raiz) e o PDF **TCC_CONTROLAAI_BD_APRESENTACAO_FINAL.pdf** (foco banco). Aqui o foco é **organização do software**.

1

Visão geral do produto

Problema · solução · canais

O **Controla.AI** é um sistema de **controle financeiro pessoal**. A ideia é simples: a pessoa fala ou escreve no **WhatsApp** (“gastei 45 no almoço”) ou usa o **painel web**, e o sistema **entende**, **categoriza**, **grava no banco** e mostra **indicadores** (ganhos, gastos, saldo, metas).

Para um leigo: imagine uma **caderneta digital inteligente**. O WhatsApp é a “porta da frente”; a **inteligência artificial (OpenAI)** é o “assistente” que lê a mensagem; o **PostgreSQL** é o “arquivo permanente”; o site na **Vercel** é o “quadro de avisos” com gráficos.

3

canais (WA · web · PDF)

1

agente IA unificado

16+

tabelas Postgres

2

clouds (Railway+Vercel)

Frontend

React + Vite + Tailwind — login, dashboard, metas, chat IA, admin WhatsApp.

Backend

Node.js + Fastify — API REST, JWT, orquestra WhatsApp e OpenAI.

Banco

PostgreSQL (Railway) via Drizzle ORM — verdade única dos dados.

IA + Zap

GPT / Whisper + Baileys — interpreta texto/áudio/foto e responde.

Regra de ouro

Tudo que importa financeiramente **passa pelo backend e termina no PostgreSQL**. O front só exhibe; o WhatsApp só entrega a mensagem.

2

Arquitetura em camadas

Apresentação → API → Domínio → Integrações → Persistência

Camada	Onde mora	Responsabilidade (em português simples)
Apresentação	frontend/src/	Telas que o usuário vê (botões, gráficos, formulários)
Edge / Proxy	frontend/api · middleware	Na Vercel: login local + encaminha /api para Railway
API REST	backend/src/	Recebe pedidos HTTP, valida JWT, chama serviços
Domínio financeiro	backend/api/	Entende texto, cria gasto, calcula KPIs, metas
WhatsApp	backend/whatsapp/	Mantém sessão do celular e entrega mensagens
Persistência	db/ + Postgres	Guarda usuários, lançamentos, logs com segurança

O arquivo **backend/src/index.ts** é o “interruptor geral”: sobe o Fastify, registra CORS, auth, rotas, billing, governança e inicia o WhatsApp. O **frontend/src/main.tsx** é o “interruptor” do site: monta React com tema e autenticação.

Princípio

Poucas pastas grandes, arquivos com **uma responsabilidade clara** e comentários em português (padrão TCC).

Documento único

TCC_DOCUMENTACAO.md na raiz — qualquer mudança de pasta/rota/schema deve atualizar esse MD.

Como as peças se ligam

Canais → Backend → Banco / OpenAI

WhatsApp (Baileys) → message-handler → financial-agent → parser / insights

Painel React → HTTPS Vercel → proxy/middleware → Fastify Railway

Ambos → **PostgreSQL** (transactions, goals, users...) · OpenAI (quando precisa interpretar)

Em produção, o navegador **não fala direto** com a URL do Railway na maior parte do tempo. Ele chama o mesmo domínio da Vercel (/api/...). O **middleware** e o **backend-proxy** leem **BACKEND_URL** e repassam o pedido para o serviço Node na Railway — isso evita CORS e esconde a URL interna.

Auth na Vercel

Login, me, forgot, reset, 2FA e settings podem rodar em **funções Node** frontend/api/auth/* (mesmo Postgres), para não depender de 502 do Railway no login.

WhatsApp só no Railway

Baileys precisa de processo **sempre ligado** (não serverless). Por isso WhatsApp vive no backend Railway/VPS, não na Vercel.

Analogia

Vercel = recepção do prédio · Railway = escritório onde o trabalho pesado (Zap + IA + CRUD) acontece · Postgres = cofre.

3

Árvore da raiz

O que cada pasta grande contém

controlaai/
├─ TCC_DOCUMENTACAO.md ← documento oficial único do TCC
├─ documentacao-tcc/ ← PDFs, PNGs ERD, este HTML
├─ frontend/ ← SPA React + funções Vercel
├─ backend/ ← API Fastify + WhatsApp + IA
├─ deploy.ps1 ← db:setup + git push (dispara CI)
├─ railway.toml ← config de build/start Railway
└─ .cursor/rules/ ← regra: atualizar TCC_DOCUMENTACAO.md

Pasta / arquivo	Para que serve
TCC_DOCUMENTACAO.md	Fonte da verdade: arquitetura, rotas, banco, histórico
documentacao-tcc/	Entrega visual (PDF BD, diagramas PNG, HTML organização)
frontend/	Interface do usuário + proxy/auth na Vercel
backend/	Cérebro do sistema: API, agente, WhatsApp, schema
deploy.ps1	Sincroniza banco e envia código ao GitHub (deploy)

Não versionar: backend/.baileys-session/ (credenciais do WhatsApp), arquivos .env com segredos, e pastas node_modules/.

Organização do backend

src · api · whatsapp · drizzle · scripts

```

backend/
├── src/                ← servidor core (Fastify, auth, CRUD, billing)
│   ├── index.ts       ← boot
│   ├── auth.ts        ← JWT, reset, OTP, 2FA
│   ├── api-routes.ts  ← extended-routes.ts / governance-routes.ts
│   ├── db/            ← Drizzle client + schema.ts
│   └── utils/         ← phone, money, admin, financial-summary
├── api/               ← agente IA, parser, insights, Stripe helpers
├── whatsapp/          ← Baileys (client, handler, bolhas, QR admin)
├── drizzle/           ← migrations SQL (0000..0013...)
├── seeds/             ← PDF/HTML TCC, seeds, migrações manuais
└── .controlaai/       ← runtime.json (modelo GPT escolhido no admin)

```

src/

É o “sistema operacional” da API: sobe servidor, segurança, rotas HTTP.

api/

É o “cérebro financeiro”: interpreta linguagem e calcula respostas.

whatsapp/

É o “telefone”: liga, recebe, responde em bolhas humanizadas.

drizzle/

É o “histórico de reformas” do banco — cada SQL altera tabelas com segurança.

Frontend e documentação

SPA + edge functions + entrega TCC

```

frontend/
├── src/
│   ├── main.tsx / App.tsx      ← boot e rotas
│   ├── pages/                  ← Dashboard, Goals, AiChat, admin...
│   ├── components/             ← Layout, guards, dialogs (ui/ = shadcn)
│   ├── lib/                    ← api.ts, auth, financial-summary
│   └── hooks/                  ← capabilities, mobile, toast
├── api/                        ← funções Node Vercel (auth, proxy, relay)
├── middleware.ts               ← decide o que proxy para Railway
└── vercel.json                 ← rewrites / build

```

documentacao-tcc/ guarda o PDF de banco, PNGs de modelagem, cópias de documentação e este arquivo HTML. Os diagramas oficiais do ER ficam em **PNGs modelagens banco dados/**.

Exceção de comentários

components/ui/* (shadcn) e seeds de demo **não** recebem comentário linha a linha — são terceiros/demo. O restante de aplicação está no mapa **MAPA-SISTEMA**.

4

Servidor Fastify — o interruptor

backend/src/index.ts

Quando a Railway sobe o container, ela executa algo como **node dist/src/index.js**. Esse arquivo: (1) carrega **env**, (2) cria o Fastify, (3) libera CORS para a Vercel, (4) registra rotas de auth/API/extended/billing/governança/WhatsApp, (5) testa Postgres e Redis, (6) garante usuário **admin@admin.com**, (7) inicia Baileys + keep-alive.

Arquivo	Função
env.ts	Lê DATABASE_URL, JWT_SECRET, FRONTEND_URL, flags Railway
redis.ts	Cliente Redis (cache/sessões) — ping no /health
db/index.ts	Pool Postgres + Drizzle
db/schema.ts	Todas as tabelas/enums em TypeScript
db/ensure-admin.ts	Cria/atualiza admin no boot

Health

A rota **/health** é o “batimento cardíaco” que a Railway usa para saber se a API está viva (banco, redis, mail).

4

Autenticação e rotas HTTP

auth · mailer · api-routes · extended · governance

auth.ts cuida de cadastro, login, JWT (~7 dias), esqueci senha, OTP por e-mail e 2FA. A senha nunca fica em texto puro — só **hash bcrypt**. **mailer.ts** envia e-mails via SMTP Gmail (local) ou **relay** na Vercel (/relay/send) quando o Railway não consegue SMTP direto.

api-routes.ts CRUD de transações, categorias, budgets, settings — o “dia a dia” do painel.	extended-routes.ts Chat IA, KPIs, metas, imports — rotas mais “inteligentes”.
governance-routes.ts Auditoria, campos LGPD, níveis de acesso, ativar/inativar.	billing-routes.ts Stripe Checkout/webhook — planos e assinaturas.

Quase toda rota financeira exige **JWT** e filtra pelo **user_id** do token — assim uma conta não vê o dinheiro de outra.

5

Agente financeiro unificado

backend/api/financial-agent.ts

Este é o **maestro**. WhatsApp e chat web chamam a mesma função de processamento. Em ordem típica: saudação? → onboarding de renda? → clarificar ganho? → fluxo de meta? → parser de gasto/consulta? → insights? → resposta padrão.

mensagem → normaliza texto → decide intenção → (grava OU consulta) → monta bolhas de resposta

Módulo auxiliar	Papel
transaction-intent.ts	Heurísticas: é gasto? receita? consulta?
goal-agent.ts	Conversa passo a passo para criar meta
onboarding-agent.ts	Pede renda mensal no início
income-classifier.ts	Renda fixa × ganho pontual
conversation-history.ts	Evita repetir a mesma frase

Por que unificar?
Mesma regra no Zap e no site = menos bug e explicação única no TCC.

5 OpenAI: parser, prompts e mídia

parser · prompts · media-processor · openai-client

parser.ts envia o texto (ou descrição de imagem/PDF) à OpenAI e espera um JSON validado pelo **Zod** (FinancialIntent): intent, valor, categoria, data... Se a IA falhar, há **fallback regex** local.

prompts.ts Textos de sistema oficiais (parser, visão, PDF) — “manual” da IA.	openai-client.ts Singleton da SDK; calcula custo aproximado de tokens.
media-processor.ts Áudio → Whisper (texto); PDF → pdf-parse.	logger.ts Grava cada chamada em ai_logs (tokens, ms, operação).

runtime-config.ts permite o admin trocar o modelo GPT no painel; salva em .controlaai/runtime.json.

6

Indicadores e consultas
insights.ts · financial-summary

Quando o usuário pergunta “quanto gastei?”, o agente não inventa: chama **insights.ts**, que consulta o Postgres e responde com números reais.

queryType	Pergunta típica	O que calcula
monthly_spending	Quanto gastei?	Soma despesas do mês
can_spend	Posso gastar 500?	Renda – gastos + projeção
health_check	Situação financeira?	KPIs agregados
biggest_expense	Maior despesa?	MAX amount
month_comparison	Vs mês passado	Dois períodos

No dashboard, **Ganhos / Gastos / Faturamento bruto / líquido** vêm da fonte única **financial-summary** (backend + espelho no front), evitando números divergentes.

7

WhatsApp — conexão

client · session · keep-alive · QR

Baileys é uma biblioteca que emula o WhatsApp Web. **client.ts** abre o socket, gera **QR Code** (lido no painel admin), salva credenciais em `.baileys-session/` e reconecta se cair. Em produção Railway, a sessão fica em volume persistente (ex.: `/data/.baileys-session`).

keep-alive.ts

A cada ~30 min verifica se o socket está saudável e reconecta.

routes.ts

API admin: status, connect, disconnect, logs do Baileys.

session-utils.ts

Resolve o caminho da pasta de credenciais.

baileys-log.ts

Buffer circular (~500 linhas) para o painel.

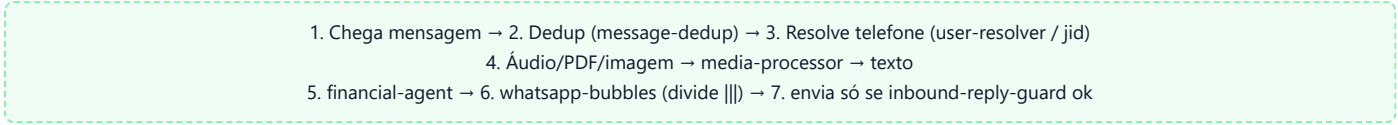
Por que não na Vercel?

Funções serverless “dormem”. WhatsApp precisa de conexão **contínua** — por isso Railway/VPS.

7

Do “oi” até a resposta

message-handler · bolhas · guards



Arquivo	Explicação leiga
user-resolver.ts	Telefone BR → usuário no banco (com/sem 9º dígito)
jid-resolver.ts	Traduz IDs LID/PN do WhatsApp novo
message-dedup.ts	Não processa a mesma mensagem duas vezes na reconexão
inbound-reply-guard.ts	Só responde se houve mensagem real (sem spam)
whatsapp-bubbles.ts	Respostas longas viram várias bolhas (mais humano)

8

Onboarding, renda e metas

onboarding-agent · income-sync · goal-*

Usuário novo: o bot pede **renda mensal** (tipo, recorrência, dia). Isso alimenta **budgets**, **user_settings** e às vezes **recurring_transactions** via **income-sync.ts** — para o painel já mostrar saldo coerente.

Metas: **goal-agent** conversa; **goal-parser** extrai valor e prazo (duration_months, deadline_at); grava em **goals** (+ checkpoints). Há proteção para não confundir “quero registrar um gasto” com meta.

- category-resolver

pizza→Alimentação, uber→Transporte — aliases locais.
- transaction-service

INSERT real em transactions + texto de confirmação.

9

React — como o site nasce

main.tsx · App.tsx · auth

main.tsx monta a árvore: ThemeProvider → AuthProvider → App. **App.tsx** define rotas (Login, Dashboard, Goals, AiChat, Settings, páginas admin) e guards (**RequireAdmin**, **RequireStaff**).

lib/auth.tsx Guarda JWT no localStorage e expõe useAuth().	lib/api.ts Cliente HTTP: em produção usa same-origin (proxy Vercel).
lib/routes.ts Decide para onde ir após login (home por perfil).	Layout.tsx Sidebar + área principal (Outlet).

O design system usa **shadcn/ui** em components/ui/ — componentes de terceiros, fora do escopo de comentário TCC.

9

Telas do produto

Dashboard · Metas · Chat · Admin

Página	O que o usuário faz
Dashboard	Vê KPIs, gráficos, lista/filtra lançamentos, abre modais
Goals	Cria/edita metas e acompanha progresso
AiChat	Conversa com o mesmo agente do WhatsApp
Settings	Perfil, 2FA, tema, export
WhatsApp (admin)	QR Code, status da sessão, modelo OpenAI
Admin*	Assinantes, auditoria, LGPD, logs de IA

Componentes de billing (**BillingPaywall**, **TrialCountdownBanner**) mostram trial/planos Stripe. **EmailOtpStep** e termos LGPD entram no cadastro.



Funções na Vercel

api/auth · proxy · relay · middleware

Além do SPA estático, a Vercel hospeda **funções Node**:

Arquivo	Papel
api/auth/login.ts · me.ts	Login/sessão direto no Postgres (menos dependência do Railway)
api/auth/forgot · reset	Fluxo esqueci senha + e-mail
api/auth-2fa-*	Enable/disable/verify/resent 2FA
api/backend-proxy/[...path].ts	Repassa /api/* para BACKEND_URL
api/relay/send.ts	Relay SMTP: Railway manda e-mail via Vercel
middleware.ts	Regras de quais paths vão ao proxy

Same-origin

O browser chama **controlaai-frontend.vercel.app/api/...**; a Vercel encaminha para **controlaai-backend-production.up.railway.app**.

10

Banco de dados

Memória permanente do sistema

O **PostgreSQL** na Railway guarda usuários, lançamentos, metas, mensagens WhatsApp, logs de IA, assinaturas Stripe e consentimentos LGPD. O código TypeScript descreve as tabelas em **schema.ts**; o Drizzle traduz isso para SQL.

16+

tabelas

18

FKs principais

1

hub: users

UUID

ids seguros

Migrations em backend/drizzle/ (0000 schema inicial até 0013 frequência de renda, etc.). Ferramentas: **db:push**, **db:migrate:all**, DBeaver para inspeção.

PDF dedicado

Detalhes de colunas/ER: **TCC_CONTROLAAI_BD_APRESENTACAO_FINAL.pdf** na pasta documentacao-tcc.

10

Principais tabelas

O que cada grupo guarda

Conta / LGPD users , user_settings, user_consent, password_reset_tokens, audit_logs	Financeiro transactions , categories, budgets, recurring_transactions
Metas goals , goal_checkpoints	Canais / IA whatsapp_messages, whatsapp_sessions, ai_logs, ai_conversations, financial_memory

CASCADE: apagar usuário remove gastos/metadados ligados. **SET NULL:** apagar categoria mantém o lançamento. Enums travam valores (expense/income, planos, etc.).

Tabela	Por que importa
transactions	Cada gasto/ganho do usuário
users.phone	Chave para achar quem mandou WhatsApp
ai_logs	Auditoria de custo/tokens OpenAI
subscriptions	Espelho do Stripe

11

Fluxo ponta a ponta — gasto no Zap

Exemplo: "gastei 45 no almoço"

1. Usuário envia no WhatsApp conectado ao Baileys
2. client.ts entrega evento → message-handler
3. Telefone → users.id (user-resolver)
4. financial-agent → parser (OpenAI ou regex)
5. category-resolver (Alimentação) → transaction-service INSERT
6. Opcional: liga whatsapp_messages.transaction_id
7. Resposta em bolhas confirma valor/categoria
8. Dashboard na Vercel, ao abrir, lê a mesma linha no Postgres

Uma verdade

WhatsApp e site **não têm bancos separados** — ambos olham o mesmo PostgreSQL da Railway.

11**Fluxo — login e dashboard**

React → Vercel → Postgres / Railway

1. Usuário abre `controlaai-frontend.vercel.app/login`
2. POST `/api/auth/login` (função Vercel ou proxy Railway)
3. Compara `bcrypt` · emite JWT
4. Front guarda token · redireciona ao Dashboard
5. Dashboard chama `/api/...` (proxy) → Fastify
6. Rotas usam `authPreHandler` · filtram por `user_id`
7. KPIs via `insights / financial-summary`

Se o 2FA estiver ligado, entra o passo OTP por e-mail antes de liberar a sessão completa.

12

Railway — API + Postgres + Redis

O "escritório" do sistema

A **Railway** hospeda o backend como serviço sempre ligado (equivalente a uma VPS gerenciada). Build TypeScript → **node dist/src/index.js**, porta pública (ex. 8080). No mesmo projeto costumam existir **PostgreSQL** e **Redis**.

Serviço	URL / papel
Backend	https://controlai-backend-production.up.railway.app
Postgres	DATABASE_URL (rede interna + URL pública SSL)
Redis	REDIS_URL — cache/sessões
Health	GET /health

railway.toml na raiz do monorepo indica como buildar/iniciar. Variáveis sensíveis ficam só no painel Railway (nunca no Git).

12

Vercel — painel e edge

SPA + middleware + functions

A **Vercel** faz o build Vite do frontend e publica em **controlaai-frontend.vercel.app**. Variável crítica: **BACKEND_URL** apontando para a URL pública Railway. **VITE_API_URL** vazio = same-origin.

O que roda na Vercel

HTML/JS do React, middleware, auth functions, proxy, relay SMTP.

O que NÃO roda

Baileys/WhatsApp contínuo, jobs longos — isso é Railway.

Push no GitHub (branch main) dispara redeploy automático na Vercel (e, se integrado, na Railway).

12

Como o código se liga ao servidor

Do arquivo .ts até a URL pública

Dev: npm run dev → localhost:3333 (API) + :5173 (Vite)

Commit/push: GitHub main

Railway: clona → npm build → sobe dist/ → escuta PORT

Vercel: clona frontend → vite build → CDN + functions

Runtime: process.env.* injetado pela nuvem (não pelo .env local)

WhatsApp: volume persistente da sessão Baileys no Railway

O arquivo **deploy.ps1** na raiz: roda db:setup, git add/commit/push — empurrando o código que as plataformas vão puxar.

Ponto de atenção

Mudou schema? rode migrations no Postgres da Railway. Mudou só front? Vercel basta. Mudou WhatsApp/agente? backend Railway precisa rebuild.

12

Variáveis essenciais

O que configurar em cada nuvem

Variável	Onde	Para quê
DATABASE_URL	Railway (+ Vercel auth)	Conexão Postgres
JWT_SECRET	Ambos	Assinar tokens de login
FRONTEND_URL	Railway	CORS e links de e-mail
BACKEND_URL	Vercel	Proxy para a API
OPENAI_API_KEY	Railway	Parser / Whisper
REDIS_URL	Railway	Cache
EMAIL_SMTP_RELAY_*	Railway+Vercel	E-mails via relay
STRIPE_*	Railway	Pagamentos
ENABLE_WHATSAPP	Railway	Liga/desliga Baileys

12

Deploy e verificação
Checklist operacional

Passo	Como validar
API no ar	GET ...railway.app/health
Front no ar	Abrir Vercel · tela de login
Proxy OK	Login + listar transações no dashboard
WhatsApp	Admin → QR / status connected
E-mail	Esqueci senha / OTP 2FA
IA	Chat ou Zap com "gastei 10 teste"

Local: **npm run dev** na raiz (backend+frontend). Produção: push main + conferir logs Railway/Vercel.

13

Mapa de arquivos comentados

Padrão TCC linha a linha em PT

Catálogos oficiais: **backend/src/MAPA-SISTEMA.ts** (BACKEND_APPLICATION_FILES) e **frontend/src/MAPA-SISTEMA.tsx** (FRONTEND_APPLICATION_FILES). Cada arquivo de aplicação traz cabeçalho **Doc TCC: TCC_DOCUMENTACAO.md** e comentários didáticos em português.

Backend (~50+)

src/*, api/* (27), whatsapp/* — exceto seeds/demo.

Frontend (~55+)

pages, lib, components app, hooks, api/*, middleware — exceto ui/*.

Manutenção

Alterou código → atualiza comentários se necessário → atualiza **TCC_DOCUMENTACAO.md** §14 histórico na mesma entrega.